

PIC RC Motor Controller (and Example Lego Robot)

By PaulMakesThings in Circuits > Remote Control  59.131  110  55 ★ Featured  Winner



Note for National Robotics Week Robot Contest: I am over 18. I am a student at University of Wisconsin Milwaukee. As a mechanical engineering student this project is related to my major. It has also taught me skills that have led to undergrad research work that is taking me on a mechatronics and robotics related career path.

What it is:

This device takes the signal from an RC receiver, the kind you would plug your servos into in an RC car or airplane. Based on that RC signal it outputs controlled power to pair of motors up to 18V and 4A. The control is proportional (variable speed), and goes in both directions without bias in either direction, making it well suited to tank drive vehicles. The logic is based on a PIC18F1320. **The best part of this project is not just that it allows you to make a relatively cheap RC dual motor controller, but that using this code and input scheme you can take any PIC microcontroller project and add radio control with as many channels as you like using only one pin to receive it all.** Besides using those values to control other robot functions you can still use any of the ports on your receiver, including the ones that are going to the control board, to control servos.

Because it's not that much fun controlling two motors without a robot for them to drive I have built the system into a Lego robot . This motor controller could be used for any skid steer robot, or you could simplify it to one motor, use the other port for a servo and make a vehicle that steers like a car. If the Lego part of the project is of interest to you the connectors are explained in step 4 and the lego body is laid out in step 5.

This project requires basic electronic tools and a programmer that can work with 18F series chips.



Смотреть на

The hardware:

The electronic hardware is fairly simple. The PIC 18F1320 handles all the signal interpretation, requiring only 2 filtering diodes. The same chip generates motor control signals for both motors. The other big chip on the board is a L298n motor driver. This chip contains two H-Bridges capable of driving a motor with up to 4A of current at up to 18V. Besides the chips the one other large component is a 5V regulator to give the PIC a good power supply. Besides those you have a handful of inexpensive diodes and resistors. A specific parts list is in step 1.

The software:

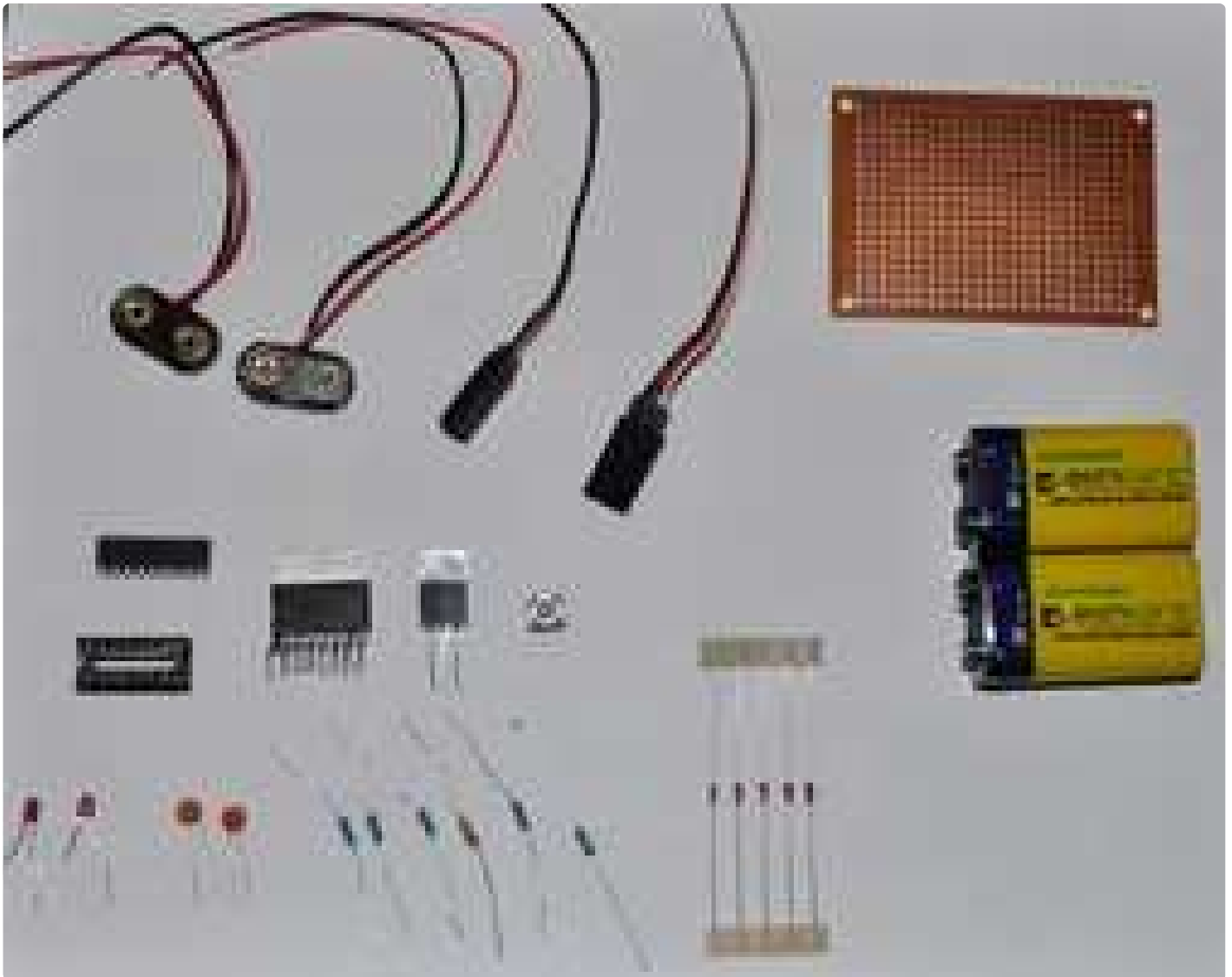
The software for the chip has a portion that interprets PWM* servo control signals into a variable, and another portion that generates two motor control PWM* signals. These parts could each potentially be used to make either a system that does something else based on radio control commands, or a motor controller that is driven by other means. The code could easily be expanded to read many (I'm talking 8 or more) PWM signals. You could take one of those huge \$500 airplane

remote control systems and have a chip read every channel! What you did with all that control data would be up to you.

A note on PWM:

These are both pulse width modulated (PWM) signals, which simply means the width of the pulses is used to represent values, but the motor control signal pulses on and off to vary the average power, or duty cycle, to accomplish various motor speeds, while the servo signal works by sending different widths of pulse which are read as relative command values.

Step 1: Parts List



General:

RC transmitter/receiver set with at least 3 channels. As is this code only works with stacked PWM signals. I can only say for sure that it will work with traxxas receivers. With modification it could work with any PWM timing, and I am working on making it work with synchronous PWM like the newer Futaba systems use, that is the pulses all begin at the same moment. It will not work with VEX rc systems, which use a different kind of signal.

since this is going to be in a SparkFun sponsored contest, links to the parts on

that website are provided, conveniently these pages also have the datasheets

Electrical

2 9V rechargeable batteries, or any other array of batteries that gives you 9 to 24 volts.

2x 9V battery connectors

A few feet of 22AWG or similar hook up wire (solid recommended if you don't know which you prefer)

Electronic components:

1x perf board with 0.1" spacing, about 2x3 inches in size

1x L298 Dual bridge driver

1x PIC18F1320

2x 0.1uF ceramic capacitors for regulator. (the code of 0.1uF is "104")

1x 18 pin IC socket (recommended but not strictly needed)

1x 5V regulator

2x small 3mm LEDs for indicators. (use a green and a red instead of two reds like i did.)

Resistors (2x 1 ohm, 2x 220 ohm, 1x 12k ohm, 1x 3.6k ohm)

10x 1N4148 or similar diodes

1x servo wire with two female ends, you can cut it in half and make both yo

Software needed (all free):

MPLAB and C18 software (free version works fine, only needed if you want to modify the code)

The plan for the board was made in the free version of EAGLE PCB.

Tools:

PIC programmer such as a PICkit 3, PICkit 2, or one of it's clones like the Junebug.

Solder

Soldering Iron with fine tip

Diagonal Cutters, or any kind of wire snipper you favor

Wire strippers

Soldering heat sink (recommended, not required)

Step 2: Software

This step will go through each part of the software and explain what it does. **If you have no interest in modifying the software and just want to build the thing, you may go on to the next step. The hex file provided will make the project work as designed.**

```
//Basic configureation of chip periferials  
#pragma config OSC = INTIO2, WDT = OFF, LVP = OFF  
#include <p18f1320.h>
```


This sets the oscillator to internal and loads the settings for the 18F1320, if you want to use a different pic this is one of the main things you will have to change

```
//setup pins for PWM input  
#define ReceiverPin PORTBbits.RB3  
#define ReceiverTris TRISBbits.TRISB3
```

This is just giving a name to the pin for receiving the signal so it is clearer to refer to.

```
//PWM capture variables  
unsigned int PWM1RiseTime = 0; //timer value at rising edge capture  
unsigned int PWM1FallTime = 0; //timer value at falling edge capture  
unsigned int PWM1Width = 0; //calculated width
```

```
unsigned int CH1_width = 0;  
unsigned int CH2_width = 0;  
unsigned int CH3_width = 0;  
unsigned int PWMGap = 0; //calculated gap between pulses
```

```
char PWM1Edge = 1; //edge currently being monitored 1 = rising, 0 = falling
```

```
//button variables  
unsigned char button_1_debounce_wait = 0;
```

Since the PWM signal from the receiver communicates by sending pulses of varying width variables are declared to hold these widths once they are calculated.

```
unsigned char calibration_mode = 0;

#define mode_operate 0
#define mode_CH1_high 1
#define mode_CH1_med 2
#define mode_CH1_low 3
#define mode_CH2_high 4
#define mode_CH2_med 5
#define mode_CH2_low 6
```

```
unsigned int limit_CH1_high = 2381;
unsigned int limit_CH1_med = 3307;
unsigned int limit_CH1_low = 4286;
```

```
unsigned int limit_CH2_high = 2022;
unsigned int limit_CH2_med = 2946;
unsigned int limit_CH2_low = 3983;
unsigned int CH_temp = 0;
```

When calibration mode is on the system will be resetting these "limits" to suit the signal. These are the defaults, how the calibration works will be explained later.

```
//motor control variables

#define Motor_PWM_Rez 16 //number if different speeds possible forward
and reverse

#define center_buffer 20 //this is the fraction of the range before movement
starts
```

These are constants you can adjust if you are using different parts. The center buffer is really the dead zone in the center where the controller wont make the

motor do anything. The resolution is how many different speeds the system will divide it's range of control into.

```
unsigned char Motor_Phase = 0; //as it cycles this will time the motors
```

```
unsigned int CH1_range = 2000;
```

```
unsigned char Motor_A_Speed = 0; //this is the speed of motor A, at %100 it  
will equal the rezolution
```

```
unsigned char CH1_forward_increment = 10; //the width of range for each  
speed output
```

```
unsigned char CH1_reverse_increment = 10;
```

```
unsigned int CH2_range = 2000;
```

```
unsigned char Motor_B_Speed = 0; //this is the speed of motor A, at %100 it  
will equal the rezolution
```

```
unsigned char CH2_forward_increment = 10; //the width of range for each  
speed output
```

```
unsigned char CH2_reverse_increment = 10;
```

```
typedef struct
```

```
{
```

```
unsigned motor_A_Direction: 1;
```

```
unsigned motor_B_Direction: 1;
```

```
unsigned button_1_last_state: 1;
```

```
}BITS;
```

```
unsigned char motor_A_inverted = 1; //this related to calibration
```

```
unsigned char motor_B_inverted = 1;
```

```
unsigned char motor_calibration_needed = 1;
```

```
volatile BITS Bits;
```

```
//timing variables
```

```
unsigned char slow_count = 0; //this is used to create the scaled timer for  
slower events
```

The variable above will just be a counter so that a subsection of the timer interrupt can go off only every one out of many timer ticks.

```
//set up interrupt
```

```
void low_ISR(void); //prototype
```

```
#pragma code low_vector = 0x08 //0X08 IS LOW 0X18 IS HIGH
```

```
void low_interrupt (void){
```

```
_asm goto low_ISR _endasm
```

```
}
```

```
#pragma code
```

```
#pragma interrupt low_ISR
```

This part isn't the interrupt in itself but it set's up for the interrupt to occur. The interrupt is just an event that allows something to be triggered so that the program doesn't have to be one big loop.

```
void main(void)
```

```
{
```

```
OSCCON = 0x72; //8MHz clock
```

```
while(!OSCCONbits.IOFS); //Wait for OSC to become stable
```

```
//configure timer1
```

```
PIR1bits.TMR1IF = 0; //clears the timer 1 interrupt flag
```

```
T1CONbits.TMR1ON = 1; //turn on timer
```

```
T1CONbits.T1CKPS1 = 0; //set prescaler
```

```
T1CONbits.T1CKPS0 = 0; //set prescaler
```

```
//setup timer2
```

```
PIR1bits.TMR2IF = 0; //clears the timer 2 interrupt flag
```

```
PIE1bits.TMR2IE = 1; //enable the interrupt
```

```
PR2 = 199;
```

```
T2CON = 0b00000100; //(-)always 0 (----) postscale (-)on/off (--) prescale
```

```
//configure CCP1
```

```
CCP1CON = 0b0000101; //configure CCP1 for capture, rising edge
```

```
INTCONbits.PEIE=1; //enable peripheral interrupts
```

```
PIE1bits.CCP1IE=1; //enabled CCP1 interrupt
```

```
INTCONbits.GIE=1; //enable branching to interrupt
```

```
ReceiverTris = 1; //set RB3 for input so the capture can work.
```

```
TRISBbits.TRISB2 = 1; //set rb2 for in so it can be used to differentiate  
channels
```

The capture module does all the heavy work here. Above it is initialized to wait for the signal to rise, later this will be changed dynamically to capture pulse width.

```
//configure ports
```

```
ADCON1 = 0xff; //all digital
```

```
INTCON2bits.RBPU = 0; //port b weak pullups on
```

```
//these will be motor outputs
```

```
TRISAbits.TRISA0 = 0;
```

```
#define Motor_Pin_A1 LATAbits.LATA0
TRISAbits.TRISA1 = 0;
#define Motor_Pin_A2 LATAbits.LATA1
TRISAbits.TRISA2 = 0;
#define Motor_Pin_B1 LATAbits.LATA2
TRISAbits.TRISA3 = 0;
#define Motor_Pin_B2 LATAbits.LATA3
```

These commands set the pins needed to control the motors to act as outputs.
Then the motor pins are named for easy access.

```
//these will be indicator outputs
TRISAbits.TRISA6 = 0;
TRISAbits.TRISA7 = 0;

//this will be the servo signal input
TRISBbits.TRISB0 = 1;

//initially calibrate the RC ranges
motor_calibration_needed = 1;
```

```
while(1)
{
}
}
```

This while loops keeps the program from ending. Don't be fooled by the fact that it is empty. Interrupts will trigger events and the timer is still running.

Below is the start of the timer interrupt. It goes off periodically at the highest speed that any function requires, for fast operations such as deciding if it's time to turn the motor on or off, then is subdivided with counters to operate the functions that do not require such high speed, such as monitoring the input and deciding if the speed needs to change.

```
void low_ISR(void)
{

//Timer 2 flag (currently set to interrupt at 10Khz)
if(PIR1bits.TMR2IF == 1)
{
PIR1bits.TMR2IF = 0; //clears the timer 1 interupt flag
```

So as to not waste time doing things faster than necessary (a good way to look at many kinds of work) the part below uses the variable "slow_count" to only execute every 100 times the outer loop executes.

```
//withen this function executes at 100Hz***
slow_count ++;
if(slow_count > 100)
{
slow_count = 1; //reset count for next time
```

```
//Handle Calibration Button
```

```
if(button_1_debounce_wait > 0){button_1_debounce_wait --;}  
if(PORTBbits.RB0 == 0){  
if(Bits.button_1_last_state == 0 && button_1_debounce_wait == 0)//button  
just pressed  
{  
button_1_debounce_wait = 10;//set debounce count  
calibration_mode++;  
if(calibration_mode > 6){calibration_mode = 0;}  
}  
Bits.button_1_last_state = 1;  
}  
else  
{  
Bits.button_1_last_state = 0;  
}  
//end of calibration button
```

Below the calibration is actually applied. This is done in normal operation mode so the lights are turned both off. The program checks if the calibration range is backwards, high is lower than low and vice versa, and if so sets a flag so that the directions of the motors will act accordingly.

```
//Handle Led Mode Indicators
```

```
if(calibration_mode == mode_operate)  
{  
LATAbits.LATA6 = 0;  
LATAbits.LATA7 = 0;
```



```
if(motor_calibration_needed == 1)
{
motor_calibration_needed = 0; //clear flag

//recalculate calibration variables for CH1
if(limit_CH1_low < limit_CH1_high)//speed increases as number increases
{
motor_A_inverted = 0;
}
else//speed decreases as number increases
{
//swap them so high is the greater value
CH_temp = limit_CH1_low;
limit_CH1_low = limit_CH1_high;
limit_CH1_high = CH_temp;

motor_A_inverted = 1;
}

CH1_range = limit_CH1_high-limit_CH1_low;
CH1_forward_increment = (limit_CH1_high-limit_CH1_med -
((limit_CH1_high-limit_CH1_med)/center_buffer))/Motor_PWM_Rez;
CH1_reverse_increment = (limit_CH1_med-limit_CH1_low -((limit_CH1_med-
limit_CH1_low)/center_buffer))/Motor_PWM_Rez;
}

//recalculate calibration variables for CH2
if(limit_CH2_low < limit_CH2_high)//speed increases as number increases
{
```

```
motor_B_inverted = 0;
}
else//speed decreases as number increases
{
//swap them so high is the greater value
CH_temp = limit_CH2_low;
limit_CH2_low = limit_CH2_high;
limit_CH2_high = CH_temp;

motor_B_inverted = 1;
}

CH2_range = limit_CH2_high-limit_CH2_low;
CH2_forward_increment = (limit_CH2_high-limit_CH2_med -
((limit_CH2_high-limit_CH2_med)/center_buffer))/Motor_PWM_Rez;
CH2_reverse_increment = (limit_CH2_med-limit_CH2_low -((limit_CH2_med-
limit_CH2_low)/center_buffer))/Motor_PWM_Rez;

}
//end of led mode indicators
```

Below calibration is handled. Each time the button is pressed the calibration mode changes, indicating that a new limit is being set. The pattern is CH1 full forward, middle resting, full backward, then the same three positions again on channel two. The light indicators show off for not in calibration mode, one on for forward, the other for reverse and both for middle resting point. It's not a robust interface but it gets the job done.

```
//calibration  
if(calibration_mode == mode_CH1_high)  
{
```

All this LATA stuff is just the lights being turned on to indicate the mode to the user. As you can see it doesn't actually set the limits when you hit the button. It just sets them at whatever spot they are while in that mode, so when you push the button again and that mode ends, that stays the calibration point.

```
LATAbits.LATA6 = 0;  
LATAbits.LATA7 = 1;  
  
limit_CH1_high = CH1_width;  
  
}  
if(calibration_mode == mode_CH1_med)  
{  
LATAbits.LATA6 = 1;  
LATAbits.LATA7 = 1;  
  
limit_CH1_med = CH1_width;  
}  
if(calibration_mode == mode_CH1_low)  
{  
LATAbits.LATA6 = 1;  
LATAbits.LATA7 = 0;  
  
limit_CH1_low = CH1_width;  
}
```

```
if(calibration_mode == mode_CH2_high)
{
  LATAbits.LATA6 = 0;
  LATAbits.LATA7 = 1;

  limit_CH2_high = CH2_width;
}
if(calibration_mode == mode_CH2_med)
{
  LATAbits.LATA6 = 1;
  LATAbits.LATA7 = 1;

  limit_CH2_med = CH2_width;
}
if(calibration_mode == mode_CH2_low)
{
  LATAbits.LATA6 = 1;
  LATAbits.LATA7 = 0;

  limit_CH2_low = CH2_width;

  motor_calibration_needed = 1;
}
```

Now the motor speeds need to be calculated. The equation gets the width of the pulse for that motor, decides if it is over the midpoint or not to decide direction, then finds it's range using the motor control resolution within the total range of possible widths.

```
//calculate motor speed A
Motor_A_Speed = 0;
if(CH1_width > limit_CH1_med+((limit_CH1_high-
limit_CH1_med)/center_buffer))//upper range
{
Motor_A_Speed = (CH1_width-limit_CH1_med -((limit_CH1_high-
limit_CH1_med)/center_buffer))/CH1_forward_increment;
Bits.motor_A_Direction = motor_A_inverted;
}
if(CH1_width < limit_CH1_med-((limit_CH1_med-
limit_CH1_low)/center_buffer))//lower range
{
Motor_A_Speed = (limit_CH1_med-CH1_width -((limit_CH1_med-
limit_CH1_low)/center_buffer))/CH1_reverse_increment;
Bits.motor_A_Direction = !motor_A_inverted;
}
```

```
//calculate motor speed B
Motor_B_Speed = 0;
if(CH2_width > limit_CH2_med+((limit_CH2_high-
limit_CH2_med)/center_buffer))//upper range
{
Motor_B_Speed = (CH2_width-limit_CH2_med -((limit_CH2_high-
limit_CH2_med)/center_buffer))/CH2_forward_increment;
Bits.motor_B_Direction = motor_B_inverted;
}
if(CH2_width < limit_CH2_med-((limit_CH2_med-
limit_CH2_low)/center_buffer))//lower range
{
```

```

Motor_B_Speed = (limit_CH2_med-CH2_width -((limit_CH2_med-
limit_CH2_low)/center_buffer))/CH2_reverse_increment;
Bits.motor_B_Direction = !motor_B_inverted;
}
//end of calculating motor speed

} //end of 100hz section

```

Here the if statement and counter that cause the above to only execute at 100Hz have ended and we are at the full timer interrupt frequency. The part below handles generating the motor control signal from the speed calculated above

```

//control pulses to motor
Motor_Phase++;
if(Motor_Phase > Motor_PWM_Rez){Motor_Phase = 1;}

//Motor A
if(Motor_A_Speed >= Motor_Phase && Motor_A_Speed < 20){
if(Bits.motor_A_Direction == 0){
Motor_Pin_A1 = 1;
Motor_Pin_A2 = 0;
}
if(Bits.motor_A_Direction == 1){
Motor_Pin_A1 = 0;
Motor_Pin_A2 = 1;
}
}
else{
Motor_Pin_A1 = 0;

```

```
Motor_Pin_A2 = 0;
}

//Motor B
if(Motor_B_Speed >= Motor_Phase && Motor_B_Speed < 20){
if(Bits.motor_B_Direction == 0){
Motor_Pin_B1 = 1;
Motor_Pin_B2 = 0;
}
if(Bits.motor_B_Direction == 1){
Motor_Pin_B1 = 0;
Motor_Pin_B2 = 1;
}
}
else{
Motor_Pin_B1 = 0;
Motor_Pin_B2 = 0;
}

} //end of timer interrupt
```

Below is the beginning of the CCP interrupt. This is the part that handles measuring the pulse width. Earlier it was set to be triggered by a rising edge. When it detects the rising edge it will record the time using CCPR1 then it will switch to watch for falling and change the PWM1Edge variable to match. When it detects falling it switches back and records the time.

```
//ccp interrupt
```

```
if(PIR1bits.CCP1IF == 1)
{
  PIR1bits.CCP1IF = 0; //clear the flag
  if(PWM1Edge == 1)//if detecting rising
  {
    CCP1CON = 0b0000100;//switch to detect falling edge
    PWM1Edge = 0;//switch to indicate falling edge is next
    PWMGap = CCPR1 - PWM1FallTime; //calculate gap between pulse starts
    PWM1RiseTime = CCPR1;//save the low timer value for the rise time

    if(PWMGap < 10000){CH2_width = PWMGap;}

  }
  else //if detecting falling
  {
    CCP1CON = 0b0000101;//switch to detect rising edge
    PWM1Edge = 1;//switch to indicate rising edge is next
    PWM1Width = CCPR1 - PWM1RiseTime; //(pwm rise time is the time that the
    pwm rise occurred)
    PWM1FallTime = CCPR1;//save the low timer value for the fall time
```

You will really need to understand the logic behind this part if you need to modify the code to work on other receivers. The traxxas receiver I used puts all of the pulses back to back. This made it so that I couldn't read through just one pin because the whole set of pulses was one long pulse when combined. So I designed the program so the chip is only hooked up to every other output, in this case servo outputs 1 and 3. That way there is a gap. The short gap (the one less than 10000 as detected by the if statement below) is the intermediate one and is

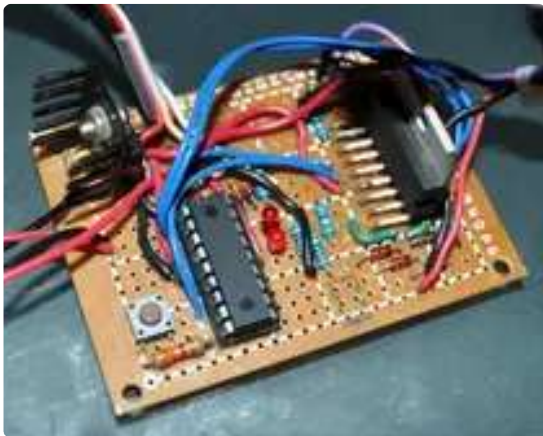
the length of the middle pulse, pulse number 2. The first pulse after the long gap is pulse number 1 and the one after the short gap is pulse number 3.

```
if(PWMGap > 10000){CH1_width = PWM1Width;}  
if(PWMGap < 10000){CH3_width = PWM1Width;}  
}  
  
}  
}
```

Please feel free to ask questions. They will help me make things clearer so I really do appreciate them.

As I mentioned in that last note this plan revolves around the pulses occurring back to back. Some receivers space them out. If that were the case you wouldn't need to do this trick at all. Instead you would just know that after the long gap was pulse one, then after each additional short gap you were looking at pulse 2, 3, 4 and so on. You would just make a variable to keep track of how many pulses you had caught since the last gap and reset it when you had the long one, then use it to decide which channel you attributed a captured pulse width to.

Step 3: The Circuit Board



I just built mine point to point on a perf board because I didn't feel it was complicated enough to require an etched circuit board. For those who do want to make a PCB the Eagle PCB files are provided, and honestly since I've never made them into a printed PCB they could probably use some improvement. By the way if anyone does make the Eagle schematic into an etched PCB I'd love to see it and have the files so I can include them here.

Attached are some schematics of components of this circuit to help explain how it works. There are only 3 main elements. The regulator, the L298N dual H-bridge motor driver and of course the PIC 18F1320. Additionally there is one push button, 2 LEDs, and a handful of resistors and protection diodes.

[This schematic](#) explains how the L298N is meant to be used. I'd post it here but I'm not sure I have permission.

Below is a simple diagram of the button wiring. The resistor to +5V is a pull up resistor. It is high resistance and without it when the button wasn't pushed the pin would be connected to nothing, so there would be no guarantee it would be low or

high, this resistor just "pulls it up" when the button connecting to 0v is not pushed.

Next to that is a diagram of the regulator. pulled from it's data sheet. It keeps the voltage at 5v for the microchips.

The full Eagle PCB schematic and board layout are available for download. A preview of the schematic is posted below as well, you will have to click on the *i* in the upper left corner and view the original image to actually read it here.

Note that the servo leads from the board should plug into channels 1 and 3 from the receiver.

Step 4: Making Connectors for the Lego Motors



If you want to wire this to Lego motors, but like me you don't want to modify your lego motors or permanently fix Lego wires onto the controller here is how you can make some modified Lego power connectors.

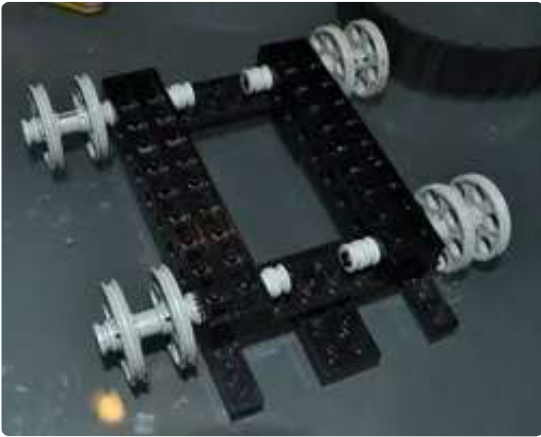
If you are simply sacrificing a good Lego wire then just cut it, split it and strip the ends, then solder on any connector you like.

I had some old Lego wires that were coming apart. If you are using a ruined Lego wire you will find that If you gently pry the edge opposite the wire back and up with a small screwdriver it will pop out. Inside you'll find that the Lego wires are soldered onto two copper plates that make contacts on the top and bottom.

Hold the part in a clamp or helping hands (the metal kind, not your friend's) and gently pull the wire while applying heat to the solder until it pulls free. Use some good flexible wire to make your replacement. Solder the ends down on the same spot where the old wires were attached since there is a space for the connection on the cover plate here. If you're worried it's too big for the cover plate to fit back

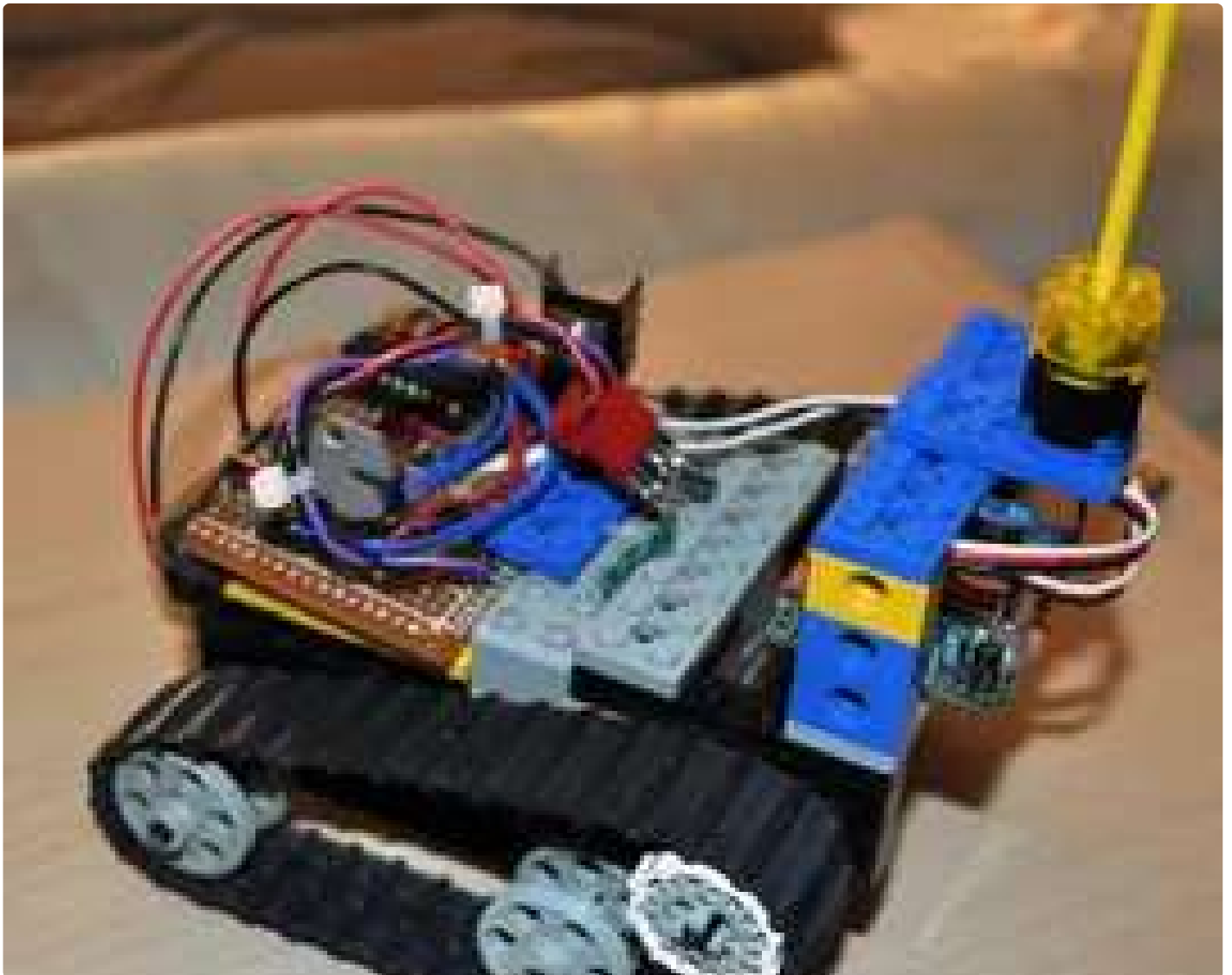
on simply press the cover plate back on immediately after soldering and it will melt out a space for itself.

Step 5: Constructing the Lego Body



For anyone interested in the Lego frame design I hope this array of photos goes through the steps.

Step 6: Other Uses



This can be used as shown to drive a tank drive vehicle, totally different motors could be used. But it has many possible alternate uses.

Car-style steering

This circuit can be built as shown but omitting the other motor. Without modifying the code the second line to the RC receiver would still be needed. Then you could connect a servo for steering to the second channel and drive the vehicle with the one motor to make a very compact solid state RC car.

Arms and actuators

If you're good with pic chips you could use the RC signals from additional channels for control. If not, you could always use channels 3 onward to control servos for anything from a simple grabber to a intricate arm.

Modifying the program

When modifying the program just remember that PWM1Width always holds the last length of high signal and PWMGap always holds the last gap. There is a huge gap between the last pulse and the first, by comparing these lengths to decide how they should be used then sorting them out the program can be extended to capture more channels or to capture pulses from receivers that put out pulses periodically rather than back to back like the Traxxas receiver used here.

Have fun and don't burn yourself on the soldering iron!